

Project Overview & Architecture

Simple English to speak | Hinglish cues to recall | Production-style design

01 PROJECT INTRODUCTION - SPEAK THIS ONCE

I built a Kafka-based order processing platform using Java 17 and Spring Boot. It has two services: Order and Inventory. The Order Service accepts requests and stores orders. The Inventory Service checks stock and sends the reservation result back. I used transactional outboxes and idempotent consumers to handle publication failures and duplicate delivery. I also added retry handling, a dead-letter topic, monitoring, and a Jenkins pipeline for local container deployment.

Bolne ka order: purpose > services > reliability > delivery.

02 CORE CONCEPTS - ONE DEFINITION EACH

Event-driven: Services react to events to perform their work.

Event aaya, tab next service ka kaam start hota hai.

Loose coupling: Services depend on event contracts rather than direct calls for this workflow.

Producer ko consumer ki immediate response ka wait nahi.

Eventual consistency: Related service states become consistent after events are processed.

Order status turant final nahi hota.

03 TECH STACK - EACH TOOL'S ROLE

MySQL + JPA/Hibernate	Durable business data and local transactions.
Kafka + 3-node KRaft	Event transport; broker replication and metadata control.
Avro + Schema Registry	Order-event schema and compatibility management.
Actuator + Micrometer	Application health and metrics exposure.
Prometheus + Grafana	Inventory metrics collection, dashboards and alerts.
Docker + Jenkins + Docker Hub	Container packaging, CI/CD and versioned images.

04 WHY KAFKA INSTEAD OF ONLY REST?

I wanted asynchronous processing so order acceptance would not wait for inventory processing. The trade-off is delayed final status and more failure-handling work.

Project Overview & Architecture

Architecture walkthrough | Follow the arrows from request to final status



05 OTHER ROUTES - KNOW THE DIFFERENCE

Duplicate event: existing eventId > skip business update > ACK safely.

Retryable technical failure: initial attempt + 2 retries, 2-second backoff; if attempts still fail, successful DLT publication routes it to orders.created.avro-dlt.

Stock kam hona business rejection hai; technical failure ka DLT route alag hai.

06 BOUNDARIES - ANSWER WITHOUT OVERCLAIMING

Delivery: at-least-once; MySQL commit and Kafka offset commit are separate.

Scope: order + inventory only; no payment, shipping or live delivery tracking.

Deployment: local Docker/Jenkins; replay is a proof-of-concept.

Source: current repository README + OrderController + Inventory processing/consumer.